Backend dokumentáció

PLANOVA – BACKEND DOKUMENTÁCIÓ

Contents

Backend dokumentáció 1
1. Áttekintés 1
2. Projektstruktúra 2
3. Konfiguráció és indítás 2
4. Adatmodell (entitások) 3
5. Felsorolás típusok (ENUMS) 6
6. API végpontok (kontrollerek) 6
7. Adatátviteli objektumok (DTO-k) 11
8. Szolgáltatás réteg (services) 14
9. Hitelesítés és jogosultságkezelés 17
*10. Valós idejű kommunikáció (SignalR) 18
11. Adatbázis konfiguráció 18
12. Hibakezelés 19
13. NuGet csomagok 20

A "*"-gal megjelölt elemek, sorok fejlesztés alatt állnak vagy jövőbeli terveket tükröznek.

1. Áttekintés

A Planova backend egy .NET 8 alapú RESTful Web API, amely a Planova naptár- és produktivitás-alkalmazás szerveroldali logikáját biztosítja.

Az API JWT-alapú hitelesítést, Entity Framework Core-on keresztüli adatbázis-kezelést, valamint *****SignalR valós idejű kommunikációt használ.

Keretrendszer: .NET 8 (C#)
Adatbázis: PostgreSQL (Npgsql provider)
ORM: Entity Framework Core 8

Hitelesítés: JWT Bearer (access + refresh token)
***Valós idejű**: Entity Framework Core SignalR
API dokumentáció: Swagger / OpenAPI (Swashbuckle)
Projektkezelés: User Secrets a szenzitív konfigurációhoz

2. Projektstruktúra

apps/backend/backendAPI/
Program.cs – Alkalmazás belépési pont és szolgáltatás-konfiguráció
appsettings.json – Alkalmazás beállítások és connection string
Controllers/ – API kontrollerek (9 db)
Models/ – Entitás modellek (9 db)
Models/Enums/ – Felsorolás típusok (5 db)
DTOs/ – Adatátviteli objektumok (modulonként csoportosítva)
Services/ – Üzleti logika és validálás (10 szolgáltatás)
Data/ – DbContext és modell konfigurációk
Data/ModelConfigurations/ – Fluent API entitás konfigurációk (9 db)
Hubs/ – SignalR hub és felhasználó-azonosító provider
Common/ – Közös hibakonstansok
Extensions/ – Controller extension metódusok
Migrations/ – EF Core migrációk (5 db)

apps/backend/backendTests/ – Egységtesztek (xUnit)

3. Konfiguráció és indítás

3.1. Adatbázis kapcsolat

A connection string az appsettings.json fájlban található, a felhasználónév és jelszó helyőrzőket ({USERNAME}, {PASSWORD}) User Secrets-ből tölti be:

Host=localhost;Port=5432;Database=Vizsgaremek;
Username={USERNAME};Password={PASSWORD};Trust Server Certificate=true

3.2. JWT konfiguráció

- Issuer és Audience: appsettings.json-ban (AppSettings szekció)
- Titkos kulcs: User Secrets-ben (jwt-secret-key)
- Token érvényesség: rövid élettartamú access token + 7 napos refresh token
- Claims: UserId (NameIdentifier), Email, ProfileId, IssuedAt, JwtId

3.3. Szolgáltatás-regisztráció

Minden üzleti szolgáltatás Scoped életciklussal regisztrálódik a DI konténerbe.
A SignalR felhasználó-provider Singleton-ként regisztrálódik.

3.4. Middleware pipeline

HTTPS átirányítás → CORS → Authentication → Authorization → SignalR hub mapping → Controller mapping

3.5. CORS

Engedélyezett origin: http://localhost:3000
Engedélyezett: minden header, minden metódus, hitelesítő adatok (credentials)

4. Adatmodell (entitások)

4.1. User

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- Email (string, unique, required) – E-mail cím
- PasswordHash (string, required) – Hash-elt jelszó
- RefreshToken (string, nullable) – Aktuális refresh token
- RefreshTokenExpiryTime (DateTime?)– Refresh token lejárati ideje
- CreatedAt (DateTime) – Létrehozás dátuma (UTC)

Kapcsolat: egy User-hez egy Profile tartozik

4.2. Profile

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- Username (string, 3-64 kar., unique) – Felhasználónév
- Avatar (string, required) – Avatar azonosító
- IsPrivate (bool) – Privát profil jelző
- FirstName (string, max 128, nullable)– Keresztnév
- LastName (string, max 128, nullable) – Vezetéknév
- BirthDate (DateOnly, nullable) – Születési dátum
- UserId (Guid, FK) – Felhasználó hivatkozás

Navigációs tulajdonságok:

- User, Habits, Calendars, SharedCalendars, CalendarEntries, EventContributors, FriendAsUser1, FriendAsUser2

4.3. Calendar

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- Name (string, 3-32 kar., required) – Naptár neve
- Color (int, 1-24) – Szín azonosító
- ProfileId (Guid, FK) – Tulajdonos profil

Navigációs tulajdonságok: Profile, CalendarEntries, SharedCalendars

4.4. CalendarEntry

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- EntryCategory (EntryCategory, required) – Bejegyzés kategória
- Name (string, 3-32 kar., required) – Bejegyzés neve
- Description (string, max 1024, nullable)– Leírás
- StartDate (DateTime) – Kezdő dátum
- EndDate (DateTime) – Záró dátum
- Location (string, nullable) – Helyszín
- NotificationTime (DateTime, nullable) – Értesítés ideje
- Color (int, nullable) – Egyedi szín (ha null: naptár színe)
- IsCompleted (bool, nullable) – Teljesítettség (feladatoknál)
- IsAllDay (bool) – Egész napos jelző
- CalendarId (Guid, FK) – Naptár hivatkozás
- CreatedBy (Guid, FK) – Létrehozó profil

Navigációs tulajdonságok: Calendar, Profile

4.5. Habit

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- Name (string, 3-64 kar., required) – Szokás neve
- Description (string, max 1024, nullable) – Leírás
- HabitCategory (HabitCategory, required) – Szokás kategória
- Unit (Unit, required) – Mértékegység
- Goal (float, nullable) – Célérték
- Color (int, 1-24) – Szín azonosító
- Days (int, nullable) – Aktív napok
- ProfileId (Guid, FK) – Tulajdonos profil

Navigációs tulajdonságok: Profile

4.6. HabitLog

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- Date (DateTime, required) – Naplózás dátuma (UTC)
- Value (float, nullable) – Mért érték
- IsCompleted (bool, nullable) – Teljesítettség
- HabitId (Guid, FK) – Szokás hivatkozás

Navigációs tulajdonságok: Habit

4.7. Friend

Mezők:

- User1ProfileId (Guid, PK, FK) – Első felhasználó profil
- User2ProfileId (Guid, PK, FK) – Második felhasználó profil
- Status (Status, required) – Barátság állapota
- CreatedAt (DateTime) – Létrehozás dátuma
- UpdatedAt (DateTime, nullable) – Utolsó módosítás

Összetett elsődleges kulcs: (User1ProfileId, User2ProfileId)
Navigációs tulajdonságok: User1Profile, User2Profile

4.8. SharedCalendar

Mezők:

- ProfileId (Guid, PK, FK) – Felhasználó profil
- CalendarId (Guid, PK, FK) – Megosztott naptár
- Role (Role, required) – Jogosultsági szint

Összetett elsődleges kulcs: (ProfileId, CalendarId)
Navigációs tulajdonságok: Profile, Calendar

*4.9. EventContributor

Mezők:

- ProfileId (Guid, PK, FK) – Felhasználó profil
- CalendarEntryId (Guid, PK, FK) – Naptárbejegyzés
- Status (Status) – Résztvevő állapota

Összetett elsődleges kulcs: (ProfileId, CalendarEntryId)
Navigációs tulajdonságok: Profile, CalendarEntry

5. Felsorolás típusok (ENUMS)

5.1. EntryCategory

- None = 0
- Task = 1
- Event = 2
- BirthDay = 3
- Anniversary = 4

5.2. HabitCategory

- None = 0
- Exercise = 1
- Reading = 2
- Mindfulness = 3

5.3. Role

- Editor = 0
- Viewer = 1

5.4. Status

- Pending = 0
- Accepted = 1
- Dismissed = 2

5.5. Unit

- None = 0
- Distance = 1
- Quantity = 2
- Time = 3

6. API végpontok (kontrollerek)

Minden kontroller az api/ útvonal-prefix alatt érhető el.
Az **[Authorize]** jelöléssel ellátott végpontok JWT hitelesítést igényelnek.

6.1. AuthController (api/Auth)

POST api/Auth/sign-up
Leírás: Új felhasználó regisztrálása

Body: SignUpRequestDTO { Email, Password }
Válasz: 200 OK / 400 BadRequest (hibaüzenet)
Logika: Tranzakcióban létrehozza a User-t, Profile-t és egy alapértelmezett Calendar-t. A felhasználónév az e-mail @ előtti részéből generálódik (egyediség biztosított).

POST api/Auth/sign-in
Leírás: Bejelentkezés
Body: SignInRequestDTO { Email, Password }
Válasz: 200 OK + TokenResponseDTO / 400 BadRequest
Logika: Jelszó ellenőrzés PasswordHasher-rel. Sikeres bejelentkezés esetén JWT access token és refresh token generálása.
A tokenek HttpOnly cookie-kban is beállítódnak (web kliens számára) és a válasz body-ban is visszatérnek (mobil kliens számára).

GET api/Auth/refresh
Leírás: Token frissítés
Forrás: Cookie-kból VAGY Authorization + X-Refresh-Token header-ből
Válasz: 200 OK + TokenResponseDTO / 401 Unauthorized
Logika: A lejárt access tokenből kinyeri a UserId-t (ValidateLifetime = false), ellenőrzi a refresh token érvényességét, majd új token-párt generál.

GET api/Auth/sign-out
Leírás: Kijelentkezés
Válasz: 200 OK
Logika: Törli az accessToken és refreshToken cookie-kat.

GET api/Auth/verify [Authorize]
Leírás: JWT érvényesség ellenőrzése
Válasz: 200 OK / 401 Unauthorized

6.2. CalendarController (api/Calendar)

GET api/Calendar [Authorize]
Leírás: Felhasználó összes naptárának lekérése
Válasz: 200 OK + GetCalendarDTO[]
Logika: UNION lekérdezéssel összevonja a saját naptárakat és a megosztott naptárakat.

POST api/Calendar [Authorize]
Leírás: Új naptár létrehozása
Body: CreateCalendarDTO { Name, Color }
Válasz: 200 OK / 400 BadRequest
Logika: Naptárnév validálás (3-32 karakter), majd mentés.

PUT api/Calendar/{calendarId} [Authorize]
Leírás: Naptár módosítása
Body: UpdateCalendarDTO { Id, Name, Color }
Válasz: 200 OK / 400 BadRequest
Logika: Jogosultság-ellenőrzés (tulajdonos vagy Editor szerepkörű megosztott felhasználó), validálás, majd frissítés.

DELETE api/Calendar/{calendarId} [Authorize]
Leírás: Naptár törlése
Válasz: 204 NoContent / 400 BadRequest
Logika: Kaszkád törlés: először a naptárbejegyzések, majd a megosztások, végül maga a naptár.

6.3. CalendarEntryController (api/CalendarEntry)

GET api/CalendarEntry/{calendarId}?startDate&endDate [Authorize]
Leírás: Naptárbejegyzések lekérdezése egy adott naptárhoz
Query: startDate (opcionális), endDate (opcionális)
Válasz: 200 OK + GetCalendarEntryDTO[] / 403 Forbid
Logika: Hozzáférés-ellenőrzés, opcionális dátumszűrés.
A bejegyzés színe: ha saját színe van, az; különben a naptár színe.

POST api/CalendarEntry/query [Authorize]
Leírás: Bejegyzések kötegelt lekérdezése több naptárból
Body: GetCalendarEntriesRequestDTO { StartDate?, EndDate?, Ids[] }
Válasz: 200 OK + GetCalendarEntryDTO[]
Logika: Hozzáférhető naptárak szűrése, majd bejegyzések lekérdezése az elérhető naptárakból.

POST api/CalendarEntry [Authorize]
Leírás: Új bejegyzés létrehozása
Body: CreateCalendarEntryDTO
Válasz: 200 OK / 400 BadRequest
Logika: Szerkesztési jogosultság ellenőrzése, naptár létezésének ellenőrzése, dátum validálás.

PUT api/CalendarEntry/{id} [Authorize]
Leírás: Bejegyzés módosítása
Body: UpdateCalendarEntryDTO
Válasz: 200 OK / 400 BadRequest

DELETE api/CalendarEntry/{id} [Authorize]
Leírás: Bejegyzés törlése
Válasz: 204 NoContent / 404 NotFound / 403 Forbid

6.4. FriendsController (api/Friends)

GET api/Friends?stat=Accepted|Pending|Dismissed [Authorize]
Leírás: Barátok listázása állapot szerint
Query: stat (alapértelmezett: "Accepted")
Válasz: 200 OK + GetFriendsDTO[]
Logika: UNION lekérdezés mindkét irányból (User1 és User2 oldalról).

POST api/Friends/friend-request [Authorize]
Leírás: Barátkérés küldése
Body: AddFriendDTO { ProfileId }
Válasz: 200 OK / 400 BadRequest
Logika: Ellenőrzi, hogy nem létezik-e már barátság, és nem saját maga-e a célszemély. Sikeres mentés után SignalR-en keresztül valós idejű értesítést küld a célfelhasználónak.

PUT api/Friends [Authorize]
Leírás: Barátság állapotának módosítása (elfogadás/elutasítás)
Body: UpdateFriendStatus { ProfileId, Status }
Válasz: 200 OK / 404 NotFound

DELETE api/Friends/{friendId} [Authorize]
Leírás: Barátság törlése
Válasz: 200 OK / 404 NotFound

*6.5. HabitController (api/Habit)

GET api/Habit [Authorize]
Leírás: Felhasználó összes szokásának lekérése
Válasz: 200 OK + GetHabitsDto[]

GET api/Habit/{habitId} [Authorize]
Leírás: Adott szokás lekérése
Válasz: 200 OK + GetHabitsDto

POST api/Habit [Authorize]
Leírás: Új szokás létrehozása
Body: CreateHabitDto { Name, Description?, Habitcategory, Unit, Goal?, Days?, Color }
Válasz: 200 OK

PUT api/Habit/{id} [Authorize]
Leírás: Szokás módosítása
Body: UpdateHabitDto
Válasz: 200 OK / 404 NotFound

DELETE api/Habit/{habitId} [Authorize]
Leírás: Szokás törlése (kaszkád: szokás naplók is törlődnek)
Válasz: 204 NoContent / 404 NotFound

***6.6. ProfileController (api/Profile)**

GET api/Profile [Authorize]
Leírás: Bejelentkezett felhasználó profiljának lekérése
Válasz: 200 OK + GetProfileDTO
Logika: A UserId claim alapján kéri le a profilt.
Tartalmazza: id, username, avatar, isPrivate, firstName, lastName, birthDate, userCreatedAt, userEmail.

PUT api/Profile/{id} [Authorize]
Leírás: Profil módosítása
Body: UpdateProfileDTO { Id, Username, Avatar, IsPrivate, FirstName?, LastName?, BirthDate?, Email }
Válasz: 204 NoContent / 400 BadRequest
Logika: Felhasználónév egyediség-ellenőrzés, profiladatok validálása (név hossz, születési dátum jövőbeli ellenőrzés).

***6.7. SharedCalendarController (api/SharedCalendar)**

POST api/SharedCalendar [Authorize]
Leírás: Naptár megosztása több felhasználóval
Body: CreateSharedCalendarDTO { Accounts[{ ProfileId, Role }], CalendarId }
Válasz: 200 OK / 400 BadRequest / 404 NotFound
Logika: Naptár létezésének ellenőrzése, majd SharedCalendar rekordok tömeges létrehozása.

PUT api/SharedCalendar/{id} [Authorize]
Leírás: Megosztott naptár jogosultságának módosítása
Body: UpdateSharedCalendarRoleDto { Role }
Válasz: 200 OK / 404 NotFound

DELETE api/SharedCalendar/{id} [Authorize]
Leírás: Megosztás törlése
Válasz: 204 NoContent / 404 NotFound

***6.8. EventContributorController (api/EventContributor)**

Jelenleg üres kontroller, implementálás tervezett.

***6.9. TestNotificationController (api/TestNotification)**

POST api/TestNotification/send-friend-request/{targetProfileId}
Leírás: Teszt végpont SignalR értesítés szimulálásához
Válasz: 200 OK
Megjegyzés: Csak fejlesztési célra, nem igényel hitelesítést.

7. Adatátviteli objektumok (DTO-k)

7.1. Auth DTOs

SignUpRequestDTO
- Email (string)
- Password (string)

SignInRequestDTO
- Email (string)
- Password (string)

TokenResponseDTO
- AccessToken (string, required)
- RefreshToken (string, required)

7.2. Calendar DTOs

CreateCalendarDTO
- Name (string, required)
- Color (int)

GetCalendarDTO
- Id (Guid)
- Name (string)
- Color (int)

UpdateCalendarDTO
- Id (Guid)
- Name (string, required)
- Color (int)

AccessibleCalendarDTO
- CalendarId (Guid)
- Role (Role enum)

7.3. CalendarEntry DTOs

CreateCalendarEntryDTO
- EntryCategory (EntryCategory enum)

- Name (string, required)
- Description (string, nullable)
- StartDate (DateTime)
- EndDate (DateTime)
- Location (string, nullable)
- NotificationTime (DateTime, nullable)
- Color (int, nullable)
- IsAllDay (bool, nullable)
- CalendarId (Guid)

GetCalendarEntryDTO
- A fenti mezők + Id, IsCompleted, CalendarName, CreatedBy, CreatedByName

UpdateCalendarEntryDTO
- A CreateCalendarEntryDTO mezői + Id, IsCompleted

GetCalendarEntriesRequestDTO
- StartDate (DateTime, nullable)
- EndDate (DateTime, nullable)
- Ids (List<string>)

***7.4. Friend DTOs**

AddFriendDTO
- ProfileId (Guid)

GetFriendsDTO
- ProfileId (Guid)
- Username (string)
- Status (Status enum)
- CreatedAt (DateTime)
- UpdatedAt (DateTime, nullable)

UpdateFriendStatus
- ProfileId (Guid)
- Status (Status enum)

***7.5. Habit DTOs**

CreateHabitDto
- Name (string, required)
- Description (string, nullable)
- Habitcategory (HabitCategory enum)
- Unit (Unit enum)
- Goal (float, nullable)

- Days (int, nullable)
- Color (int)

GetHabitsDto
- A fenti mezők + ProfileUsername, HabitCategory

UpdateHabitDto
- A fenti mezők + Id

***7.6. Profile DTOs**

CreateProfileDTO
- Username (string, required)
- Avatar (string, nullable)
- IsPrivate (bool)
- FirstName, LastName (string, nullable)
- BirthDate (DateOnly, nullable)
- UserId (Guid)

GetProfileDTO
- Id (Guid)
- Username, Avatar, IsPrivate
- FirstName, LastName, BirthDate
- UserCreatedAt (DateTime)
- UserEmail (string)

UpdateProfileDTO
- Id (Guid)
- Username, Avatar (string, required)
- IsPrivate (bool)
- FirstName, LastName, BirthDate
- Email (string, required)

***7.7. SharedCalendar DTOs**

CreateSharedCalendarDTO
- Accounts (List<SharedCalendarAccount>)
- ProfileId (string)
- Role (Role enum)
- CalendarId (Guid)

GetAllSharedCalendarDto
- ProfileUsername (string)
- CalendarName (string)
- Role (Role enum)

UpdateSharedCalendarRoleDto
- Role (Role enum)

7.8. Generikus válasz

ServiceResponse<T>
- Data (T, nullable) – Visszatérési adat
- Success (bool) – Sikeres-e a művelet
- Message (string?) – Hibaüzenet sikertelen esetben

8. Szolgáltatás réteg (services)

Minden szolgáltatás interfész + implementáció párosként van megvalósítva, és Scoped életciklussal regisztrálódik a DI konténerbe.

8.1. AuthService (IAuthService)

Felelősség: Felhasználó hitelesítése és token-kezelés

Metódusok:

- SignUpAsync(SignUpRequestDTO) → ServiceResponse<Guid>
  E-mail formátum és jelszó erősség validálás, duplikált e-mail ellenőrzés, jelszó hash-elés (PasswordHasher<User>), mentés.
- SignInAsync(SignInRequestDTO) → TokenResponseDTO?
  Felhasználó keresése e-mail alapján, jelszó verifikáció, JWT access token és refresh token generálás.
- RefreshTokensAsync(TokenResponseDTO) → TokenResponseDTO?
  Lejárt access tokenből UserId kinyerése, refresh token validálás az adatbázisból, új token-pár generálás.

Belső segédmethods:

- CreateToken() – JWT token generálás (claims: UserId, Email, ProfileId, IssuedAt, JwtId)
- GenerateRefreshToken() – 32 bájtos kriptográfiai random token
- GenerateAndSaveRefreshTokenAsync() – Refresh token mentés DB-be (7 napos lejárat)
- GetUserIdFromExpiredToken() – UserId kinyerése lejárt tokenből
- ValidateRefreshTokenAsync() – Refresh token és lejárat ellenőrzés
- IsValidEmail() – E-mail formátum validálás
- IsValidPassword() – Jelszó szabályok ellenőrzése (min. 8 karakter, kis- és nagybetű, szám, speciális karakter, max 128 karakter)

8.2. UserRegistration (IUserRegistration)

Felelősség: Felhasználó regisztrálása tranzakcióban

Metódusok:

- RegisterUserWithProfileAndCalendarAsync(SignUpRequestDTO)
  → ServiceResponse<bool>
  Adatbázis-tranzakcióban:
    1. User létrehozása (AuthService.SignUpAsync)
    2. Profile létrehozása (felhasználónév az e-mail @ előtti részéből)
    3. Alapértelmezett Calendar létrehozása
       Hiba esetén: rollback.
- GenerateRandomUniqueUsername(email) → string
  Egyedi felhasználónév generálás az e-mail címből.

8.3. CommonValidationService (ICommonValidationService)

Felelősség: Általános validálási műveletek

Metódusok:

- EntityExists<T>(Guid id) → ServiceResponse<T?>
  Generikus entitás-létezés ellenőrzés.
- ValidateEnum<TEnum>(string value) → ServiceResponse<TEnum>
  Enum érték validálás és parse-olás.
- ValidateText(string, minLength?, maxLength?) → ServiceResponse<bool>
  Szöveg hossz validálás (alapért. 3-32 karakter).

8.4. CalendarValidationService (ICalendarValidationService)

Felelősség: Naptár CRUD műveletek validálása

Metódusok:

- ValidateCalendarCreation() – Név validálás
- ValidateAndGetCalendarForUpdateAsync() – Létezés, jogosultság és név validálás
- ValidateAndGetCalendarForDeletionAsync() – Létezés és jogosultság
- ValidateCalendarEditingPermissionAsync() – Szerkesztési jogosultság (tulajdonos VAGY Editor szerepkörű megosztott felhasználó; Viewer szerepkör esetén elutasítás)
- HasCalendarAccessAsync() – Hozzáférés ellenőrzés (tulajdonos VAGY bármilyen megosztás)
- GetAccessibleCalendarsAsync() – Elérhető naptárak listája szerepkörrel

8.5. CalendarEntryValidationService (ICalendarEntryValidationService)

Felelősség: Naptárbejegyzés műveletek validálása

Metódusok:

- ValidateCalendarEntryCreationAsync() – Szerkesztési jogosultság, naptár létezés, dátum validálás
- ValidateAndGetCalendarEntryForUpdateAsync() – Létezés és validálás

Validálási szabályok:

- Csak Task kategóriájú bejegyzés lehet „teljesített"
- A záró dátum nem lehet korábbi a kezdő dátumnál

***8.6. ProfileValidationService (IProfileValidationService)**

Felelősség: Profil műveletek validálása

Metódusok:

- ValidateProfileCreationAsync() – Felhasználónév, születési dátum, userId validálás
- ValidateProfileUpdateAsync() – Egyedi felhasználónév ellenőrzés, profil létezés, közös szabályok
- ValidateUniqueUsername() – Felhasználónév egyediség

Validálási szabályok:

- Felhasználónév: 3-64 karakter
- Születési dátum: nem lehet jövőbeli

***8.7. FriendValidationService (IFriendValidationService)**

Felelősség: Barátság műveletek validálása

Metódusok:

- ValidateFriendCreationAsync() – Duplikált barátság és önbarátság ellenőrzés
- ValidateAndGetFriendForUpdateAsync() – Létezés és validálás
- FriendExists() – Barátság keresés mindkét irányban

***8.8. HabitValidationService (IHabitValidationService)**

Felelősség: Szokás műveletek validálása

Metódusok:

- *ValidateHabitCreationAsync() – (még nem implementált)
- ValidateHabitUpdateAsync() – Létezés és név validálás

*8.9. SharedCalendarValidationService (ISharedCalendarValidationService)

Felelősség: Megosztott naptár műveletek validálása

Metódusok:

- SharedCalendarCreationAsync() – Duplikált megosztás ellenőrzés
- FindSharedCalendar() – Megosztott naptár keresés

9. Hitelesítés és jogosultságkezelés

9.1. Token kezelés

Access token (JWT):

- Claims: UserId, Email, ProfileId, IssuedAt, JwtId
- Aláírás: SymmetricSecurityKey (HMAC)
- Validálás: Issuer, Audience, Lifetime, IssuerSigningKey
- ClockSkew: Zero (pontos lejárat)

Refresh token:

- 32 bájtos RandomNumberGenerator → Base64 string
- Az adatbázisban (User tábla) tárolódik
- 7 napos lejárat
- Rotáció: minden frissítéskor új refresh token generálódik

9.2. Token megszerzés sorrendje

A backend az alábbi sorrendben keresi a tokent:

1. accessToken cookie (webes kliens)
2. SignalR query string access_token (/notifications útvonalra)
3. Authorization: Bearer header (Swagger/natív kliens)

9.3. Jogosultsági szintek naptáraknál

- Tulajdonos: minden művelet engedélyezett
- Editor (megosztott): olvasás + szerkesztés engedélyezett
- Viewer (megosztott): csak olvasás engedélyezett

9.4. Controller extension metódusok

- GetUserId() – UserId kinyerése a ClaimTypes.NameIdentifier claim-ből
- GetProfileId() – ProfileId kinyerése az egyedi "ProfileId" claim-ből

*10. Valós idejű kommunikáció (SignalR)

10.1. NotificationHub

Útvonal: /notifications
A hub osztály üres, a funkcionalitást az IHubContext<NotificationHub>
biztosítja a kontrollerekben (server-to-client push).

10.2. CustomUserProvider

Az IUserIdProvider implementációja, amely a SignalR kapcsolatokat
a JWT token „ProfileId" claim-je alapján azonosítja. Ez teszi lehetővé,
hogy a Clients.User(profileId) hívás a megfelelő felhasználónak
küldje az üzenetet.

10.3. Jelenleg implementált értesítések

Barátkérés értesítés (FriendsController):
Esemény neve: "ReceiveNotification"
Payload: { Type, Name, Message, SentAt }

11. Adatbázis konfiguráció

11.1. AppDbContext

DbSet-ek: Users, Profiles, Calendars, SharedCalendars,
CalendarEntries, Habits, HabitLogs, Friends, EventContributors

A modell konfigurációk Fluent API-val és annotációkkal vannak definiálva, az
Assembly.GetExecutingAssembly()-ból automatikusan betöltődnek
(ApplyConfigurationsFromAssembly).

11.2. Modell konfigurációk (IEntityTypeConfiguration)

9 konfiguráció a Data/ModelConfigurations/ mappában:

- CalendarConfig
- CalendarEntryConfig
- EventContributorConfig
- FriendConfig
- HabitConfig

- HabitLogConfig
- ProfileConfig
- SharedCalendarConfig
- UserConfig

11.3. Migrációk

5 migráció:

- 2026.01.15 – Initial (kezdeti séma)
- 2026.01.23 – MinorFixes
- 2026.01.23 – SecondMinorFixes
- 2026.01.28 – ThirdMinorFixes
- 2026.01.29 – NullableCalendarEntryColor (bejegyzés szín nullable-re)

12. Hibakezelés

12.1. Közös hibaüzenetek (Common/)

AuthErrors:

- InvalidEmail – „Invalid email format."
- InvalidPassword – Részletes jelszó-szabályok leírása
- UserAlreadyExists – „User with this email already exists."
- InvalidCredentials – „Invalid email or password."

CommonErrors:

- InvalidRoute – „Route ID and body ID do not match"
- ImATeapot – „You are not allowed to contribute to this calendar"

12.2. ServiceResponse<T> minta

Minden szolgáltatás ServiceResponse<T>-t ad vissza:

- Success = true/false a művelet eredménye
- Message tartalmazza a hibaüzenetet
- Data tartalmazza a visszaadott adatot

A kontrollerek a ServiceResponse alapján döntik el a HTTP válaszkódot (200/201/204/400/401/403/404).

12.3. Konkurens módosítás kezelés

Az Update műveleteknél (Calendar, CalendarEntry, Profile, Habit, SharedCalendar) DbUpdateConcurrencyException elkapása történik. Ha az entitás törlődött a módosítás közben, 404 NotFound válasz.

13. NuGet csomagok

- **Microsoft.AspNetCore.Authentication.JwtBearer 8.0.22**
- **Microsoft.EntityFrameworkCore 8.0.22**
- **Microsoft.EntityFrameworkCore.Design 8.0.22**
- **Microsoft.EntityFrameworkCore.Tools 8.0.22**
- **Microsoft.EntityFrameworkCore.Relational 8.0.22**
- **Microsoft.AspNetCore.SignalR 1.2.9**
- **Npgsql.EntityFrameworkCore.PostgreSQL 8.0.11**
- **Swashbuckle.AspNetCore 6.6.2**
- **System.IdentityModel.Tokens.Jwt 8.15.0**
- **Microsoft.VisualStudio.Web.CodeGeneration.Design 8.0.22**